Create a GitHub OAuth App

First, you need to create an OAuth application using GitHub. This process provides the client ID and client secret that the Food Finder application requires to connect to GitHub. If you do not already have a GitHub account, create one at https://github.com and log in. Next, navigate to https://github.com/settings/developers and create a new OAuth app in the **OAuth Apps** section.

Use **Food Finder** as the application name, set the homepage URL to http://localhost:3000, and set the authorization callback URL to http://localhost:3000/api/auth/callback/github.

- **Create** or **log in to** a GitHub account.
- **Navigate** to GitHub's Developer Settings and open the OAuth Apps section.
- **Register** a new OAuth application.
- **Enter** the application name as *Food Finder*.
- **Set** the homepage URL to http://localhost:3000.
- **Set** the authorization callback URL to /api/auth/callback/github.
- **Save** the application and **record** the generated client ID and client secret.

Add OAuth Credentials to the App

Now copy these credentials into the env.local file located in the root of the application's code directory. Add them using the environment variable names GITHUB_CLIENT_ID and GITHUB_CLIENT_SECRET.

- **Open** the env.local file in the project root.
- **Add** entries for GITHUB_CLIENT_ID and GITHUB_CLIENT_SECRET.

Install and Configure Authentication

To add Auth.js's OAuth SDK for **next-auth** to the Food Finder app and configure it to connect to the OAuth provider, install the package using Docker. Run the installation command inside the application container.

By default, this SDK uses encrypted JSON Web Tokens (JWTs) to store and attach session information to API requests. The library automatically handles encryption and decryption if you provide it with a secret. Use any secret you'd like.

- **Install** the next-auth package inside the application container.
- **Generate** a secret string for session encryption.
- **Add** NEXTAUTH_SECRET to the env.local file.

Create the Authentication Callback Route

Now you develop the /api/auth route that corresponds to the authorization callback URL you provided to GitHub when you registered the OAuth application. To do this, you create an auth folder inside the pages/api directory and add a file named [...nextauth].ts. The ellipsis in the filename tells the Next.js router that this file is a *catch-all route*, meaning it handles all API requests under /auth, such as /auth/signin and /auth/callback/github.

Inside this file, you configure authentication using GitHub as the OAuth provider. You import the required dependencies, including the built-in GitHub provider from next-auth and Node's crypto module. You then define a helper function that takes a string—such as a user's email address—and returns a hashed value. This hashed value will serve as a stable internal user ID.

Next, you create and export the default asynchronous authentication handler. In this function, you initialize next-auth and register the GitHub provider, configuring it with the client ID and client secret you stored in environment variables earlier.

Because you want to keep the application stateless, you rely on JSON Web Tokens (JWTs) and session callbacks. These callbacks run whenever next-auth creates or updates a JWT or session. Within these callbacks, you check whether a hashed user ID already exists. If it does not, generate one from the user's email address and store it as a private claim in both the token and the session object.

1. Set up the authentication file

- **Create** an API route named [...nextauth].ts in pages/api/auth. This is a catch-all route so this single file can handle all authentication-related endpoints, such as sign-in and callback requests.

2. Import the required dependencies

- **Import** the GitHubProvider from next-auth/providers/github.
- **Import** NextAuth from next-auth.
- **Import** NextApiRequest and NextApiResponse types to properly handle API requests.

- **Import** the createHash function from crypto, Node's crypto tools so you can securely hash user data.

3. Create a helper function to generate a user ID

- **Define** a function that accepts a string value (such as an email address).
- **Hash** that string using a secure algorithm (SHA-256).
- **Store** the hashed value in a constant so you have a stable internal user ID. You will use this value as a private identifier instead of exposing the user's email.

4. Create and export the authentication handler

- **Define** an asynchronous API handler function.
- **Initialize** NextAuth inside this function.
- **Pass** the request and response objects to NextAuth so it can manage cookies, tokens, and redirects.
- **Pass** another object containing a **providers** and a **callbacks** attribute.
- **Providers** is an array containing the GithubProvider object which takes as an argument an object with 2 attributes, clientId and clientSecret. Provide each attribute with the appropriate value.
- **Callbacks** is an object containing two asynchronous functions.
- **The first** is called jwt and has one parameter, a token. This function should
    - check the token for an email attribute and make sure the token does not contain a fdlst_private_userID attribute.
    - If both of those conditions are met, set the fdlst_private_userID attribute to the result of the hash generation function you created and give it the token.email attribute as an argument.
    - Then return the token
- **The second** is called session it will take a session object as a parameter.
    - Check If session.user.email contains a value and session.user.fdlst_private_userId does not.
    - Set the value of session.user.fdlst_private_userId to the result of your createUserId function using the session.user.email as an argument.

By doing this, you introduce a new property—fdlst_private_userId—on the session's user object. As expected, TypeScript warns you that this property does not exist on the default Session type. To resolve this, you update the custom TypeScript declaration file (customs.d.ts) in the project's root directory.

- Import mongoose so you can reference and manage the shared database connection.
- Import DefaultSession from next-auth so you can safely extend the existing session type.

- Declare a global variable for mongoose to let TypeScript recognize the reused database connection on the global object.
- Declare the module for next-auth to customize its built-in types.
- Extend the Session interface inside that module.
- Redefine the user object on the session to include a new fdlst_private_userId property for your internal user ID.
- Merge this new property **&** the default session user fields so existing session data remains available.

In this file, you extend the Session interface provided by next-auth. You import the default session type and explicitly redeclare the user object to include the new fdlst_private_userId property while preserving the existing user fields. This ensures that TypeScript recognizes the custom session property throughout the application.

Share Sessions Across the App

After setting up the callback URL, you need to make the user's session available across all Next.js pages and React components. To do this, you use the context functionality provided by next-auth. In the pages/_app.tsx file, you wrap the application in a SessionProvider so session data can be accessed throughout the app.

- Import the global and layout CSS files so styles apply across the entire application.
- Import the AppProps type from Next.js to type the custom App component correctly.
- Import the shared Layout component so all pages use the same layout.
- Import SessionProvider from next-auth/react to manage authentication state.
- Define a custom App component that receives the page component and its props.
- Extract the session object from pageProps while preserving the remaining props.
- Wrap the application in a SessionProvider and pass in the session.
- Render the Layout component inside the provider.
- Render the active page component inside the layout, passing along the remaining props.

You import the SessionProvider from the next-auth package and add the session object to pageProps. You pass the current session into the provider so it is available throughout the Next.js application. Before you can use session data in the frontend or middleware, you add the auth element with a **Sign In** button so users can authenticate.